

ML4N: SSH Shell Attack Session

MARCO DI BERNARDO, MICHELE PIO LASALVIA, STEFANO LIBERATI, and DAVIDE ROTONDO, Politecnico di Torino, Italy

This report presents a comprehensive analysis of 235,035 malicious SSH sessions captured within a honeypot environment, utilizing machine learning to automate intent classification and pattern recognition. We introduce a robust pre-processing pipeline that incorporates Base64 payload decoding and TF-IDF feature extraction to structure raw command logs. In the supervised learning phase, Random Forest and Multi-Output Logistic Regression models are evaluated, achieving weighted F1-scores exceeding 99% in classifying MITRE ATT&CK tactics. To uncover latent attack campaigns, we apply unsupervised clustering using K-Means and GMM on SVD-reduced features, which identifies five distinct behavioral groups, successfully isolating specific resource hijacking campaigns and botnet activities from generic reconnaissance. Finally, we implement a transfer learning approach by fine-tuning a BERT architecture. The deep learning model demonstrates exceptional precision comparable to the statistical baselines (F1-score ≈ 0.99), validating the robustness of contextual embeddings in characterizing complex, multi-stage attack sessions.

CCS Concepts: • **Security and privacy** → **Intrusion detection systems**; *Network security*; • **Computing methodologies** → *Supervised learning*; *Neural networks*; Unsupervised learning.

Additional Key Words and Phrases: SSH Attacks, Honeypot, Intrusion Detection, Machine Learning, Clustering, BERT, Natural Language Processing

1 Introduction

In the current landscape, SSH is the standard protocol for server access, as most systems consist of Linux-based containers or Virtual Private Server without a graphical interface. We selected this project because it mirrors real-world operational scenarios. Leveraging machine learning to detect cyber-attacks is not only a compelling and complex challenge, but it also significantly outperforms manual log analysis in terms of speed and efficiency when identifying anomalous patterns.

The dataset includes 235,035 entries, each representing a complete session of commands executed by attackers within a honeypot environment. The report is organized into four main parts: section 2. Exploratory Data Analysis; section 3. Supervised Learning Experiments; section 4. Clustering Techniques; section 5. Neural Networks Implementations.

2 Data exploration and pre-processing

In this section, we will analyze the dataset containing all the SSH attack sessions. Through systematic data processing and temporal analysis, we characterize the timing of attacks. By analyzing the seasonal distribution, we identify specific trends where malicious activity is most concentrated.

2.1 Data exploration

The temporal analysis (Figure 1 and Figure 2) reveals three distinct operational phases: a dormant period (June–Oct 2019), a massive organized surge peaking in December (**>6,000 attacks/day**, totaling **77,000 for the month**), and an erratic ‘pulse’ phase in early 2020 where activity remained unstable but elevated.

Cyclical patterns (Figure 3 and Figure 4) strongly suggest automated scheduling. Weekly activity is consistent but slightly favors the start of the week (**Mon/Tue >35,000**). The daily distribution exhibits a distinct ‘crescent’

Authors’ Contact Information: Marco Di Bernardo, marco.dibernardo@studenti.polito.it; Michele Pio Lasalvia, michele.lasalvia@studenti.polito.it; Stefano Liberati, stefano.liberati@studenti.polito.it; Davide Rondo, davide.rondo@studenti.polito.it, Politecnico di Torino, Torino, Italy.

pattern: volume peaks during late-night hours (9 PM – 2 AM) and reaches a minimum in the afternoon (3–4 PM), confirming that attacks follow a scripted rather than interactive timeline.

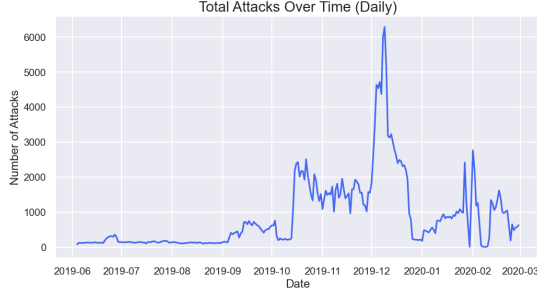


Fig. 1. Number of attacks frequencies over time.

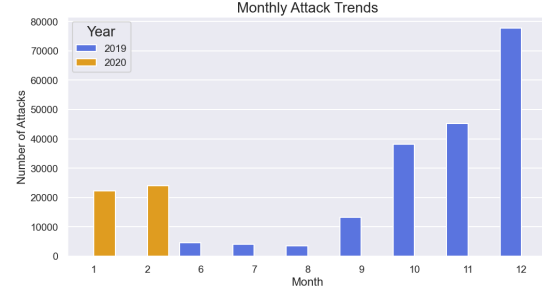


Fig. 2. Monthly attack trends.

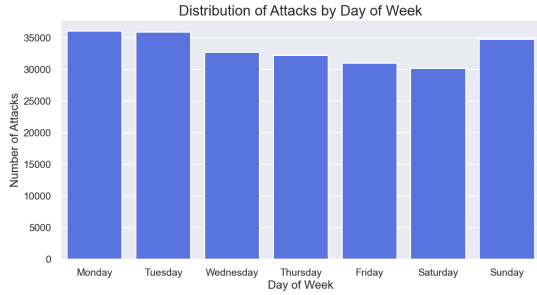


Fig. 3. Distribution of attacks by day of the week.

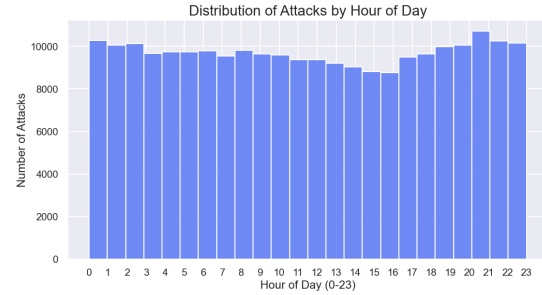


Fig. 4. Distribution of attacks by hour of day.

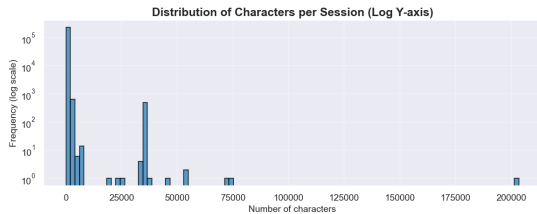


Fig. 5. Distribution of character per session.

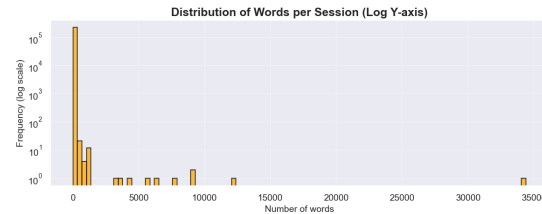


Fig. 6. Distribution of words per session.

The distribution of characters and words (Figure 5 and Figure 6) exhibits identical, heavily skewed patterns dominated by low-volume interactions ($> 10^5$ sessions). By utilizing a logarithmic scale, we isolate a long tail of rare outliers reaching **200,000 characters** and **35,000 words**. Beyond standard brief interactions, the data reveals distinct secondary clusters, specifically between **30,000–50,000 characters** and **5,000–10,000 words**, indicating a specific subset of sessions involving extensive content transfer. For a deeper statistical analysis of these distributions, refer to Appendix A.

Building on the session volume analysis, the "Top 30 Most Common Words" (Figure 7) provides a clear window into the actual commands and intent behind the recorded SSH attacks.

The data reveals a heavy focus on system reconnaissance and file manipulation, which is characteristic of automated brute-force or post-compromise scripts:

- The most frequent tokens are standard Linux utilities like **grep**, **cat**, and **echo**, with “grep” alone appearing nearly 1.3×10^6 times. These are typically used to search for sensitive information within configuration files or system logs once access is gained.
- There is a significant volume of commands aimed at identifying the target’s hardware and environment, such as **uname**, **/proc/cpuinfo**, and **model**. This suggests attackers are profiling systems to deploy specific, compatible malware.
- The frequent appearance of **rm -rf** indicates attempts to delete logs or system files to cover tracks or cause disruption. Furthermore, the use of **sleep** and hidden directory paths like **.x17-unix** points toward automated scripts designed to remain persistent and evade immediate detection.
- Tokens such as **/var/tmp/.var03522123** and **/tmp/up.txt** highlight the use of temporary directories as staging areas for downloading and executing malicious payloads.

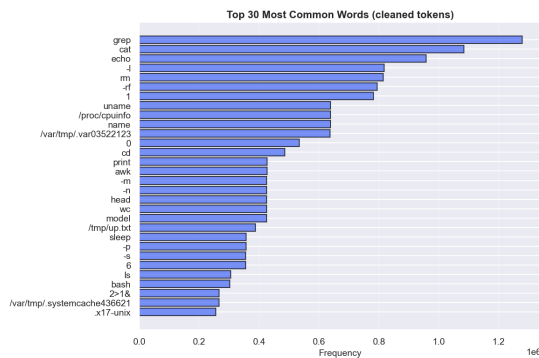


Fig. 7. Top 30 most common words.

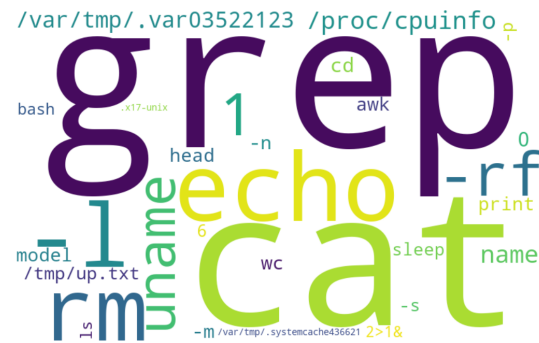


Fig. 8. Word Cloud of the 30 most common words.

The Word Cloud of the 30 most common words (Figure 8) serves as a powerful visual summary that reinforces the frequency data found in the previous histogram.

The analysis of the Distribution of Number of Intents per Session provides a clear view of the complexity of the recorded SSH attacks. The vast majority of attack sessions (Figure 9) are relatively simple, with 2 or 3 intents being the most frequent behaviors, occurring in over 10^5 and 10^4 instances respectively. As the number of intents increases, the frequency drops significantly on a logarithmic scale. Sessions with 4 to 7 intents are rare, representing more sophisticated or multi-stage manual intrusions. Interestingly, sessions with only 1 intent are less common than those with 2 or 3, suggesting that most automated scripts are designed to perform at least a couple of distinct actions.

The classification of malicious behavior (Figure 10) into high-level tactical goals reveals a clear hierarchy in the lifecycle of these SSH attacks. The distribution is heavily weighted toward Discovery and Persistence, both of which exceed 10^5 occurrences, indicating that the primary objective of most automated sessions is system profiling and establishing a long-term foothold. This is closely followed by Execution and Defense Evasion, reflecting a systematic progression from reconnaissance to the deployment of malicious scripts while simultaneously attempting to bypass security controls. Conversely, more destructive or high-noise intents like Impact appear

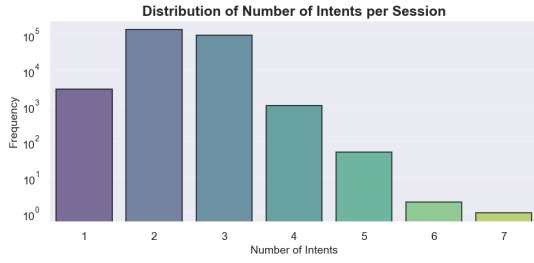


Fig. 9. Distribution of number of Intents per session.

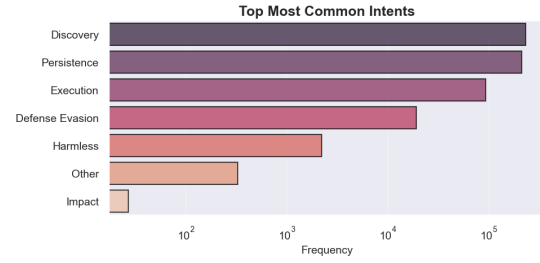


Fig. 10. Top most common intents.

with significantly lower frequency, suggesting that attackers prioritize clandestine data gathering and botnet integration over immediate system disruption.

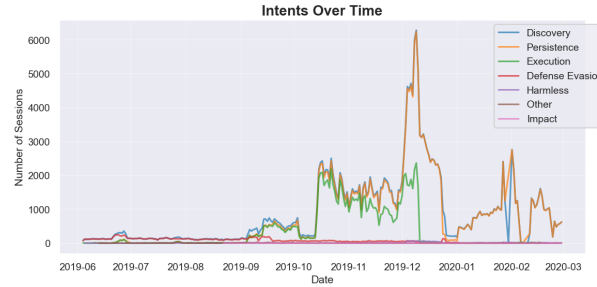


Fig. 11. Intents over time.

The temporal analysis of attack intents (Figure 11) reveals that the strategic focus of the malicious activity shifted significantly throughout the monitored period. While activity remained at a low, stable baseline for the first several months, a massive surge in Discovery and Persistence intents began in late 2019, peaking in December with over 6,000 sessions per day. This spike was closely mirrored by a rise in Execution intents, suggesting a coordinated campaign that moved rapidly from system profiling to active payload deployment. Following this peak, the volume of discovery-based sessions dropped sharply in early 2020, while persistence-related activity showed a notable secondary resurgence in February, indicating that attackers were likely attempting to maintain or re-establish control over previously compromised systems rather than seeking new targets.

2.2 Pre-Processing: Refining Data Quality

The primary dataset consists of 230,035 unique Unix shell attack instances captured during a honeypot deployment. Each session is categorized based on the specific command sequences executed, and the dataset is structured into five distinct informational columns.

- **session_id**: an integer identifying a session.
- **full_session**: the text related to the full attack session, i.e., the sequence of commands and their parameters (strings).
- **first_timestamp**: when the attack started. (e.g, 2019-06-04 09:45:50.396619+00:00).
- **Set_Fingerprint**: For each attack session, there is a set of intents associated with it. The set is a subset of {'Persistence', 'Discovery', 'Defense Evasion', 'Execution', 'Impact', 'Other', 'Harmless'}.

The “full_session” attribute records raw shell commands executed during each SSH session, serving as a primary source for identifying specific attack vectors. Due to the unstructured nature of these logs, text pre-processing is essential to extract meaningful features and convert them into a numerical format suitable for both supervised and unsupervised learning. During this phase, we identified significant portions of Base64 encoded strings within the sessions. Decoding these revealed hidden Bash commands, suggesting an intentional use of obfuscation to mask malicious activities. A detailed description is provided in Appendix B.

2.3 Pre-Processing: Data transformation

To prepare the dataset for both supervised and unsupervised learning, the raw command logs must be transformed into a structured numerical format. This process involves converting the text into a Bag of Words (BoW) representation, which serves as the foundational step for subsequent TF-IDF (Term Frequency-Inverse Document Frequency) weighting.

2.3.1 Bag of Words (BoW). The initial transformation converts qualitative text into a mathematical matrix where the frequency of each command is recorded. A specialized `CountVectorizer` is employed using a non-whitespace token pattern (`r'\S+'`) to ensure that technical artifacts, such as directory paths and command flags, are preserved as single entities.

The feature space is constrained to the top 1,000 most frequent tokens and filtered through a custom list of “stop words” to eliminate non-informative session noise, focusing the model on high-signal behavioral indicators.

The resulting BoW matrix represents each attack session as a vector, providing a quantitative basis for analyzing the tactical surges and shifts in intent observed throughout the monitoring period.

2.3.2 Term Frequency-Inverse Document Frequency (TF-IDF). To refine the numerical representation for machine learning, the TF-IDF (Term Frequency-Inverse Document Frequency) algorithm was applied to the pre-processed session data. While the previous Bag of Words (BoW) approach simply counted occurrences, TF-IDF weighs tokens based on their informational value, effectively distinguishing between routine system noise and unique attack signatures.

The TF-IDF transformation converts the cleaned sessions into a weighted matrix by calculating a score for each term t in a document (session) d within the total dataset D . The score is derived from the product of two distinct statistics:

- Term Frequency ($TF(t, d)$): this measures how frequently a command (t) appears in a single session (d).

$$TF(\text{word}, \text{session}) = \frac{\text{Number of times the word appears in the session}}{\text{Total number of words in that session}} \quad (1)$$

- Inverse Document Frequency ($IDF(t, D)$): This measures how rare a term is across the entire dataset, calculated as:

$$IDF(\text{word}, \text{session}) = \log \left(\frac{\text{Total number of sessions } N}{1 + \text{Number of sessions containing the word}} \right) \quad (2)$$

The final weight is the product of these two values ($TF * IDF$). This methodology focuses the analysis on the most tactically significant behaviors observed during the monitoring period. By calculating the IDF, generic commands that appear in almost every session receive a score near zero. This prevents automated “noise” from skewing the results of the supervised learning models. Unique strings associated with Persistent or Execution, such as specific hidden directory names or encoded payloads, receive the highest weights due to their rarity across the broader dataset. The resulting weighted matrix is serialized and stored, allowing these importance scores to be reused across different supervised and unsupervised tasks. This ensures that the time-consuming computation of importance weights does not need to be repeated during model training.

3 Supervised Learning

In this chapter, various models for classifying attack session tactics are trained. Each session can have multiple labels. Initially, the dataset is split to facilitate a training phase using default parameters, after which performance is evaluated. Following this, hyperparameters are tuned to train other models to observe performance changes. Finally, results are discussed, and different feature combinations are tested. To simplify the comparison of model performance, all results and metrics are stored in a Pandas DataFrame and exported to an **Excel file**.

3.1 Dataset split

To prepare the data for training, we utilized `MultiLabelBinarizer` to transform the target fingerprints into a binary matrix, where each column represents one of the seven attack tactics. We then partitioned the TF-IDF feature matrix (233,035 samples) into training (80%) and testing (20%) sets using a random state for reproducibility. This resulted in **186,428 samples** for model optimization and **46,607 samples** reserved exclusively for final evaluation.

3.2 Training of two models

The Random Forest Classifier was selected as the first model due to its robustness and capability to handle **multi-label classification** tasks effectively in high-dimensional feature spaces. The model was trained using the feature matrix defined in subsection 3.1.

Regarding performance evaluation, the accuracy was computed by comparing predicted labels against ground truth labels for both the training and test sets. The model achieved an accuracy of 99% on the training set and 98% on the test set.

However, the classification report (Table 1) on the test set reveals varying performance depending on the attack tactic. The most represented classes, such as *Discovery*, *Execution*, and *Persistence*, exhibit excellent scores. Conversely, minority classes present significant challenges. The *Harmless* class is particularly problematic: while precision is high (92%), recall is critically low (18%). This indicates that 82% of actual *Harmless* instances are misclassified (False Negatives). Finally, metrics for the *Impact* class (only 2 samples) are not statistically significant.

Table 1. Classification Report (Test) - Random Forest

Class	Precision	Recall	F1-Score	Support
Defense Evasion	1.00	0.97	0.98	3,760
Discovery	1.00	1.00	1.00	46,433
Execution	1.00	0.99	0.99	18,512
Harmless	0.92	0.18	0.31	458
Impact	1.00	0.50	0.67	2
Other	1.00	0.99	0.99	67
Persistence	1.00	1.00	1.00	42,281
<i>micro avg</i>	1.00	0.99	1.00	111,513
<i>macro avg</i>	0.99	0.80	0.85	111,513
<i>weighted avg</i>	1.00	0.99	1.00	111,513

For a visual breakdown of misclassifications per class, particularly regarding the False Negatives in minority intents, refer to the **confusion matrices** in Appendix C.

The minimal difference between training (99%) and test (98%) accuracy suggests the absence of significant overfitting; the model generalizes well on majority classes. The primary limitation remains **class imbalance**.

Since the Random Forest algorithm tends to prioritize majority classes to maximize global accuracy, this issue motivates the strategies applied in the next section regarding Hyperparameter Tuning and Cross-Validation.

The **Logistic Regression** (wrapped in a **MultiOutputClassifier**, because this strategy transforms the complex multi-label problem into many simple binary problems that Logistic Regression can solve) was chosen as the second model because it is particularly effective for high-dimensional sparse data, such as TF-IDF vectors. The model was trained using the same feature matrix.

Regarding metrics, the model achieved an accuracy of 98% on both training and test sets, showing no signs of overfitting. However, the classification report (Table 2) reveals results similar to Random Forest. While represented classes have excellent scores, minority classes suffer. The *Harmless* class shows high precision (98%) but very low recall (13%), meaning 87% of actual *Harmless* instances are missed.

Table 2. Classification Report (Test) - Logistic Regression

Class	Precision	Recall	F1-Score	Support
Defense Evasion	0.99	0.97	0.98	3,760
Discovery	1.00	1.00	1.00	46,433
Execution	1.00	0.99	0.99	18,512
Harmless	0.98	0.13	0.23	458
Impact	0.00	0.00	0.00	2
Other	1.00	0.99	0.99	67
Persistence	1.00	1.00	1.00	42,281
<i>micro avg</i>	1.00	0.99	1.00	111,513
<i>macro avg</i>	0.85	0.72	0.74	111,513
<i>weighted avg</i>	1.00	0.99	0.99	111,513

As before, the corresponding **confusion matrices** are detailed in Appendix C.

In conclusion, Both models exhibit comparable predictive performance, but they share the same weakness regarding minority classes. However, Logistic Regression proves to be computationally more efficient, being approximately seven times faster than Random Forest.

Finally, regarding generalization, the analysis reveals **no significant overfitting** for either model, as evidenced by the negligible gap between training and test accuracies ($< 1\%$). This stability confirms that both algorithms generalize well to unseen data, although their high aggregated performance is primarily driven by the dominant attack classes.

3.3 Hyper-parameters Tuning

To do this optimization in terms of the classification accuracy and analyzing model stability, a hyper-parameter tuning step using GridSearchCV was conducted. The main objective of this step is to check if optimized hyper-parameters can improve minority class identification (*Harmless* and *Impact*) over default values used in the baseline model. To avoid high computation time during model grid search, a subsampling approach with a 3-fold Cross-Validation technique on a reduced set of 15,000 samples was used. Model parameters were optimized based on Weighted F1-score; hence, optimized models were re-fit on the entire dataset and evaluated using a separate Test Set.

For Random Forest Classification, hyper-parameter tuning emphasized a balance between model complexity and a good level of generalization capacity. Hyper-parameters were optimized using a grid search with the following combinations:

- **n_estimators** {30, 50, 100}: To check if a smaller number of trees can deliver competitive performance.
- **max_depth** {10, 20, *None*}: To control overfitting.
- **min_samples_leaf** {1, 2}: To make a margin among decisions.

Hyper-parameter tuning produced results with maximum complexity parameters (`n_estimators = 100`, `max_depth = None`, `min_samples_leaf = 1`) indicated that a deep model using deep trees can better grasp TF-IDF space intricacies.

For MultiOutput Logistic Regression, tuning emphasized regularization and class imbalance modeling:

- **C** {0.1, 1, 10}: Inverse of regularization strength.
- **class_weight** {None, 'balanced'}: To provide automatic class weights based on frequency.

The outcomes of the Grid Search (visualized as heatmaps in Appendix C) confirm the robustness of the baseline configurations. As shown in the supplementary heatmaps, increasing model complexity yielded negligible performance gains, indicating that the default parameters were already near-optimal for this feature set. The optimized hyper-parameters were obtained: `C = 10`, indicating a need to fit models very closely despite regularization, and `class_weight = None`.

3.3.1 Performance Comparison and Analysis. The results highlight a strong robustness of the baseline configurations (Table 3). The comparison shows that the optimized models do not significantly outperform the baseline ones, indicating that the original hyperparameters were already well suited for this specific task. This behavior can be attributed to the distinctive nature of the command-based features, which allow the models to achieve near-perfect accuracy for most intent classes, such as *Discovery* and *Persistence*, regardless of further parameter tuning.

Table 3. Comparison of Best Hyper-parameters and Performance Metrics

Model	Best Hyper-parameters Found	Val. F1 (W)	Test F1 (W)	Test F1 (M)
Random Forest	n_estimator: 100 max_depth: None min_samples_leaf: 1	0.9947	1.00	0.85
Logistic Regression	C: 10 class_weight: None	0.9939	1.00	0.75

Another important aspect is the strong consistency between validation and test performance. The Validation F1-scores of 0.9947 for the Random Forest model and 0.9939 for Logistic Regression are closely aligned with the Test Weighted F1-score of 1.00. This consistency suggests that both models generalize well to unseen data and do not suffer from overfitting, reinforcing their reliability in real-world scenarios.

However, a noticeable discrepancy emerges when comparing Weighted F1 and Macro F1 scores. While the Weighted F1 reaches a perfect score of 1.00, the Macro F1 drops to 0.85 for Random Forest and 0.75 for Logistic Regression. This gap is primarily caused by the extremely low representation of *Impact* and *Harmless* samples in the dataset. Although the models classify most traffic samples with almost perfect accuracy, they struggle to achieve complete recall on these minority classes, which significantly affects the Macro F1 metric.

Finally, even after hyperparameter tuning, Random Forest maintains a clear advantage over Logistic Regression, particularly in terms of Macro F1-score. This result further confirms the superior ability of Random Forest to handle the complexity and multi-label characteristics of SSH session data, making it a more effective choice for this classification task.

3.4 Result for each intent

A granular analysis (Table 1 and Table 2) reveals a performance dichotomy driven by class balance. The dominant tactics (*Discovery*, *Persistence*, *Execution*) are identified with near-perfect F1-scores (≈ 1.00), confirming that their command signatures (e.g., `wget`, `grep`) are mathematically distinct in the TF-IDF space. Conversely, minority classes suffer from extreme imbalance. The **Harmless** class (Support: 458) exhibits a "conservative" bias: high precision (>0.92) but critically low recall (<0.18), indicating that the models struggle to distinguish benign outliers from the mass of malicious traffic. **Impact** (Support: 2) is effectively undetectable by Logistic Regression (Recall 0.00), though Random Forest captures 50% of instances, showcasing the benefits of non-linear decision trees for edge cases.

3.5 Feature Exploration and Ablation Study

To better understand which factors influence the model's performance and to assess whether a more complex TF-IDF representation is necessary, an ablation study was conducted. Starting from the full model, two simplified feature configurations were evaluated: a Bag of Words representation, which relies solely on raw term frequencies without IDF weighting, and a Metadata Only configuration, which completely ignores command text and uses only two scalar features per session, namely the total number of characters and the total number of words.

Table 4. Comparison of Model Performance (Weighted F1-Score) across different Feature Sets

Feature Set	Random Forest	Logistic Regression
TF-IDF (Baseline)	0.9965	0.9960
Bag of Words (BoW)	0.9965	0.9964
Metadata Only	0.9961	0.9894

The outcome of this comparative analysis can be viewed in Table 4. The comparison between TF-IDF and Bag of Words reveals an almost complete equivalence in performance. In the case of Random Forest, both representations achieve the same F1-score (0.9965), indicating that the addition of IDF weighting does not provide any measurable advantage. For Logistic Regression, Bag of Words performs marginally better, with an improvement of 0.0004. This slight difference can be attributed to the IDF penalty applied to very common terms such as `grep` or `cd`, which, although frequent, are strong indicators of the majority class (*Discovery*) in this dataset. Penalizing them therefore slightly reduces their usefulness. Overall, these results suggest that the relative weighting of terms plays a minor role, while the mere presence or absence of key commands is sufficient to discriminate between attack intents. The attack patterns are so distinct that a simple frequency-based representation already captures the necessary information for near-perfect classification.

Even more surprising is the performance achieved by the Metadata Only configuration. Using only session length measured in characters and words, the Random Forest model reaches an F1-score of 0.9961, which is virtually indistinguishable from the text-based approaches. This result indicates a strong correlation between the structure of a session and its underlying intent. The fact that such minimal metadata can yield such high accuracy suggests that the dataset contains highly repetitive and automated attack behaviors. In this context, the "shape" of the traffic, namely session length, is almost as distinctive as the command content itself. However, relying solely on metadata would be problematic in a real-world scenario, as an adaptive attacker could easily manipulate session length by padding commands without changing the actual payload, thereby evading detection.

Although TF-IDF was ultimately selected as a robust and content-aware baseline, this ablation study demonstrates that, for this specific dataset, the predictive information is largely redundant. The exceptionally strong performance achieved using only basic session statistics indicates that the attacks follow predictable templates

and are executed in a fully automated manner, making them trivial to classify regardless of the sophistication of the feature representation.

4 Unsupervised Learning

Before applying clustering algorithms to uncover hidden patterns within SSH sessions, an additional preprocessing step was required to address the high dimensionality and sparsity of the TF-IDF feature space, which originally consisted of 233,035 samples and 1,000 features. High-dimensional spaces are well known to negatively affect the efficiency and reliability of distance-based algorithms such as K-Means. This phenomenon, commonly referred to as the *Curse of Dimensionality*, causes Euclidean distances to lose discriminative power as dimensionality increases, making clustering results less meaningful.

To mitigate this issue, a dimensionality reduction strategy was adopted. Instead of applying standard Principal Component Analysis (PCA), which would be computationally expensive and inefficient given the sparse nature of TF-IDF representations, Singular Value Decomposition (SVD) was employed. SVD is particularly well suited for sparse text-based data and allows for efficient projection into a lower-dimensional space. Through this approach, the dimensionality was reduced from 1,000 to 100 components, achieving a cumulative explained variance of 82.66%. This result indicates that a substantial portion of the original variance, and thus the underlying structure of the attack commands, was preserved, while a large amount of noise and non-informative variation was effectively removed.

Finally, although clustering is inherently an unsupervised learning task and does not require labeled data, ground truth labels were prepared for evaluation purposes. These labels were derived from the fingerprint column using a MultiLabel Binarizer and were intentionally excluded from the clustering process itself. They were reserved exclusively for external evaluation which will be presented in subsection 4.5. This separation ensures an objective assessment of clustering quality and enables a meaningful comparison with known MITRE ATT&CK tactics through external validation metrics such as Purity and the Adjusted Rand Index (ARI).

To ensure a robust identification of attack patterns and strictly validate the discovered structures, we've employed two complementary clustering algorithms: **K-Means** and **Gaussian Mixture Models (GMM)**. K-Means was selected as the baseline deterministic approach, favoring spherical and compact clusters. In contrast, GMM was implemented to capture more complex, probabilistic structures, allowing for elliptical cluster shapes and soft assignments.

Finally, while both algorithms were evaluated quantitatively, the detailed semantic characterization (subsection 4.4) focuses primarily on the K-Means results. Since the GMM analysis yielded largely congruent behavioral patterns, its specific semantic breakdown and word cloud visualizations are reported in Appendix D, while the quantitative comparison between the two methods is discussed in subsection 4.5.

4.1 Determining the Optimal Number of Clusters

To analyze the underlying structure of the unlabeled dataset and determine the optimal number of clusters (k), an iterative analysis using K-Means was performed on the reduced feature space obtained via SVD. The analysis explored values of k in the range $[2, 15]$ and employed two complementary assessment methods:

- **Clustering Error (Inertia):** This metric evaluates cluster compactness by calculating the Sum of Squared Errors (SSE) between data points and their respective centroids. The goal is to identify the “elbow point,” beyond which increasing k provides only marginal improvements in cluster compactness.
- **Silhouette Coefficient:** This metric assesses clustering quality by comparing intra-cluster cohesion to inter-cluster separation. Higher values indicate better-defined, more clearly separated clusters.

The upper bound of the k range was set to 15 based on heuristic reasoning derived from the available Ground Truth. The dataset contains 7 known tactics from the MITRE ATT&CK framework, which serve as an initial

estimate for the number of high-level behavioral groups. To allow the algorithm to explore potential sub-structures within these tactics (e.g., different operational patterns within the Discovery tactic), the upper limit was doubled to 14 and slightly extended to 15. This ensures sufficient flexibility for detecting sub-clusters while avoiding over-segmentation, which could lead to noisy or poorly defined clusters.

The results, summarized in Figure 12, show the following trends:

- **Clustering Error (Inertia):** The inertia curve exhibits a steep initial decline, with a clear inflection point between $k = 4$ and $k = 5$. Beyond $k = 5$, the decrease in error becomes marginal, indicating diminishing returns from adding additional clusters.
- **Silhouette Score:** The silhouette score provides a more decisive indication. It increases gradually and reaches a global maximum of 0.7751 at $k = 5$. Increasing k to 6 causes a sharp drop (approximately 0.64), suggesting that adding another cluster disrupts the cohesion of already well-formed clusters.

Based on this evidence, $k = 5$ was selected as the optimal number of clusters. While the Elbow method suggests that a plausible range could be between 4 and 5, the silhouette analysis clearly identifies $k = 5$ as the point of maximal cluster separation and cohesion. Values beyond this threshold are likely to introduce noise without meaningful gains in clustering quality.

We've conducted a parallel analysis using a Gaussian Mixture Model that unlike K-Means, which assumes spherical clusters and minimizes variance, is a probabilistic model that maximizes the likelihood of the data given the model parameters. We analyzed the Log-Likelihood score across the same range of clusters ($k \in [2, 15]$). The results exhibited a similar trend to the K-Means analysis. As shown in Figure 13, the Log-Likelihood curve displays a sharp increase up to $k=4$ and $k=5$, after which the rate of improvement diminishes significantly.

Although the Silhouette Score for GMM peaked at $k=3$, the Log-Likelihood metric strongly supports a higher complexity model. The "elbow" in the Log-Likelihood curve at $k=5$ confirms that five components provide the most parsimonious fit for the data's underlying probability distribution. This cross-verification confirms that the intrinsic dimensionality of the behavioral patterns is consistent regardless of whether a hard-partitioning (K-Means) or soft-probabilistic (GMM) approach is used.

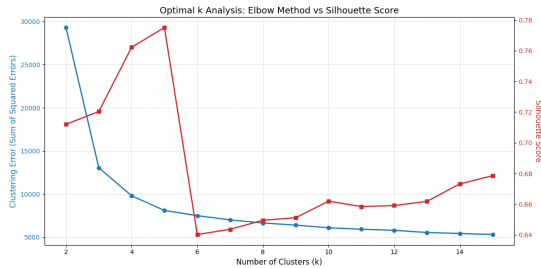


Fig. 12. Determination of the optimal number of clusters for K-Means using Elbow Method and Silhouette Score.

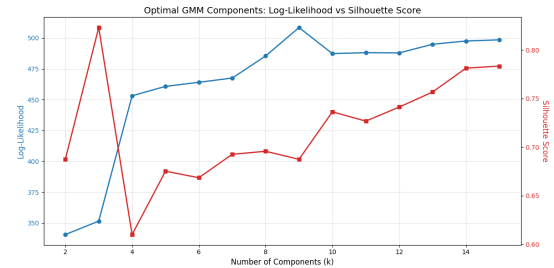


Fig. 13. Determination of the optimal number of clusters for GMM using Log-Likelihood and Silhouette Score.

4.2 Hyper-parameter Tuning

After identifying the optimal number of clusters ($k=5$), the next step focused on fine-tuning the internal parameters of the K-Means algorithm. The goal was to verify whether the clustering results corresponded to a global minimum and whether the clusters had reached a stable configuration. A Grid Search was performed, targeting three key parameters:

- **Initialization (init):** Comparison between the traditional k-means++ initialization, which spreads centroids strategically, and random initialization.
- **Number of Restarts (n_init):** Evaluation of 10 versus 20 restarts, to avoid convergence to a local minimum.
- **Algorithm:** Comparison between the standard Lloyd algorithm and Elkan, which can be more computationally efficient for dense data but may consume more memory.

Table 5. K-Means Clustering Performance Comparison with different Initialization Strategies

Initialization	n_init	Algorithm	Silhouette Score	Clustering Error (Inertia)
k-means++	10	Lloyd	0.7751	8101.8
k-means++	20	Lloyd	0.7751	8101.8
random	20	Lloyd	0.6213	9337.37
k-means++	10	Elkan	0.7751	8101.8

The tuning process was conducted on a validation subset of 15,000 samples, balancing computational efficiency with reliability. The results are summarized in Table 5.

A clear performance degradation is observed when using random initialization, despite an increased number of restarts. Compared to k-means++, random initialization results in a significantly lower Silhouette Score and higher clustering error, indicating convergence to a sub-optimal local minimum.

This behavior highlights the presence of local minimum in the clustering landscape and demonstrates that convergence quality is strongly influenced by centroid initialization. In contrast, all configurations using k-means++ consistently converge to the same optimal solution, regardless of the number of restarts or the algorithm variant. This confirms the robustness of the clustering structure in the reduced SVD feature space when appropriate initialization is employed.

The results also show that increasing n_init from 10 to 20 does not provide additional benefits when using k-means++, suggesting that convergence to the global minimum is achieved reliably within 10 restarts.

Based on these findings, the final configuration was automatically selected as the best-performing one according to the Silhouette Score maximization criterion applied during the Grid Search.

Among all tested configurations, the combination `init='k-means++'`, `n_init=10`, and `algorithm='Lloyd'` consistently achieved the highest Silhouette Score while minimizing clustering error. Configurations relying on random initialization converged to sub-optimal local minima, despite an increased number of restarts.

To validate the effectiveness of the hyper-parameter optimization, the performance of the tuned model was compared against the baseline configuration used in subsection 4.1.

Table 6. Performance Comparison between Baseline and Tuned K-Means Models

Model Version	Configuration	Inertia	Silhouette Score
Baseline	n_init=10, init=k-means++	8102.0	0.7751
Tuned	n_init=10, init=k-means++	8101.8	0.7751

The results in Table 6 show that the baseline and tuned models achieve effectively identical performance. This indicates that the baseline configuration had already converged to the optimal clustering solution.

Following the K-Means optimization, a parallel Grid Search was conducted for the Gaussian Mixture Model to identify the configuration that best balances cluster separation (Silhouette Score) and model fit (Log-Likelihood). The search targeted three parameters:

- **Covariance Type:** comparing full (each cluster has its own general shape), tied (all clusters share the same shape), and diag (clusters are axis-aligned ellipses).
- **Initialization:** comparing kmeans vs. random.
- **Restarts (n_init):** to assess stability.

The results, summarized in Table 7, reveal a distinct trade-off. The tied covariance type achieved the highest Silhouette Score (0.8598), significantly outperforming the full and diag variants. This suggests that the attack clusters in the projected space share similar geometric properties (e.g., similar density and orientation). By constraining the model to learn a single shared covariance matrix, we likely reduced overfitting, resulting in cleaner separation boundaries.

Table 7. GMM Hyper-parameter Tuning Results

Covariance	Init	n_init	Silhouette	Log-Likelihood
tied	kmeans	1	0.8598	274.87
diag	kmeans	1	0.6819	464.68
full	kmeans	1	0.6751	460.75
full	random	5	-0.0450	496.51

Ultimately, the configuration `covariance_type='tied', init='kmeans', n_init=1` was selected as the optimal GMM model, as it maximized the primary clustering quality metric (Silhouette) and demonstrated superior separation even compared to the optimized K-Means model (0.86 vs 0.78).

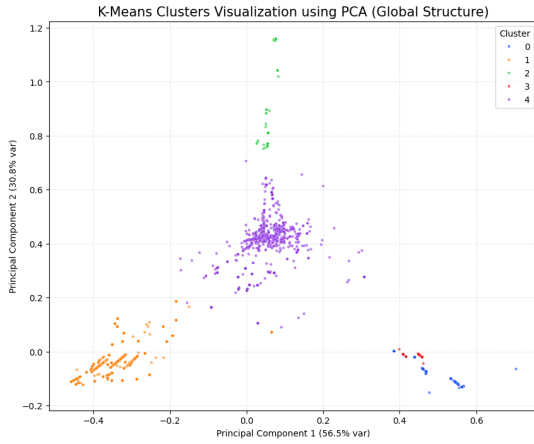
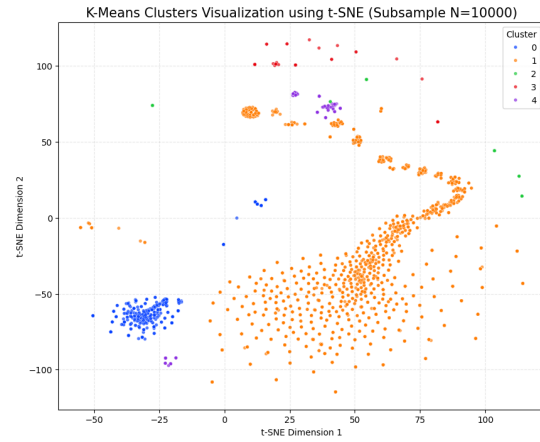
4.3 Visualization of Clusters

To gain an intuitive geometric understanding of the discovered behavioral patterns and to validate the stability of the partition, two complementary dimensionality reduction techniques were employed. By projecting the 100-dimensional feature space into two dimensions, we assessed whether the five identified clusters correspond to genuine structural separations in the data or merely artifacts of a specific algorithm. This visual validation was performed for both the **K-Means** (Figure 14 and Figure 15) and **GMM** (Figure 16 and Figure 17) partitions.

4.3.1 Global Structure Analysis (PCA). Principal Component Analysis (PCA) was first applied to visualize the global variance structure. The resulting projection (PC1: 56.5%, PC2: 30.8%) reveals that the data points do not collapse into a single amorphous cloud. Instead, a clear **radial topology** emerges, characterized by a dense central core and three primary elongated branches extending outward (Top, Bottom-Left, and Bottom-Right). Crucially, this structural organization is consistent across both clustering algorithms, though the probabilistic nature of GMM offers a more nuanced view of the boundaries between these regions. For clarity, the cluster indices referenced in the following analysis correspond to the K-Means model:

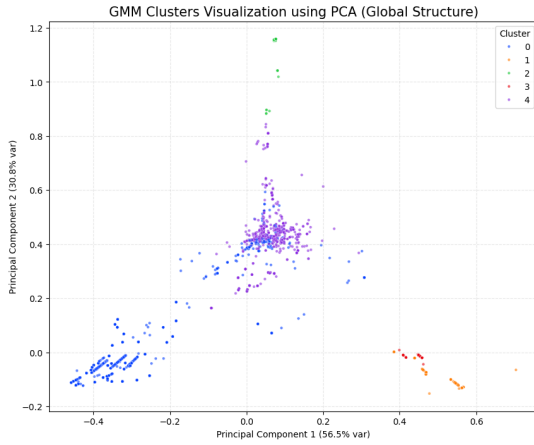
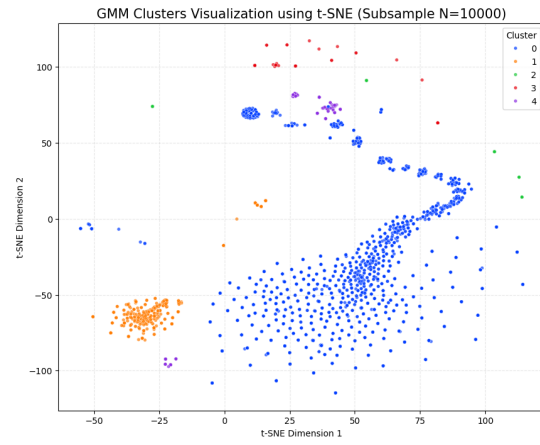
- **The Central Core (Cluster 4):** The center of the projection is dominated by **Cluster 4**. In the K-Means projection, this core is sharply delineated and well-separated from the branches. However, the GMM analysis reveals that this center is not a closed system. Instead, GMM assigns the outer edges of this core, specifically the points transitioning toward the Top and Bottom-Left branches, to the neighboring clusters. This suggests that **Cluster 4** represents a generic "Staging" state that naturally evolves into specific attack campaigns, rather than a strictly isolated behavior.

- **The Top Branch (Cluster 2):** The branch extending to the top is assigned to **Cluster 2**. While K-Means treats this as a discrete unit, GMM identifies a blending effect at the base of the branch, where the specific attacks of Cluster 2 merge probabilistically with the generic central commands of Cluster 4.
- **The Bottom-Left Branch (Cluster 1):** The branch extending to the bottom-left corresponds to **Cluster 1**. Here, the difference between the algorithms is most pronounced. While K-Means draws a hard line separating this branch from the center, GMM identifies a continuous transition zone, grouping the base of this branch with the central core. This indicates a strong variance gradient, suggesting these attacks are direct, high-volume extensions of the staging commands.
- **The Bottom-Right Subdivision (Clusters 0 and 3):** The third branch, extending to the bottom-right, exhibits the most complex internal structure. Notably, **both K-Means and GMM subdivide this single geometric region into two adjacent clusters** (identified as **Cluster 0** and **Cluster 3** in the K-Means model). The fact that both algorithms independently agree on splitting this branch, despite their different mathematical assumptions, strongly suggests that this region contains two genuine, distinct variations of the same tactical family that are mathematically separable.

Fig. 14. PCA projection of K-Means clusters ($k = 5$).Fig. 15. t-SNE visualization of K-Means clusters ($k = 5$).

4.3.2 Local Manifold Analysis (t-SNE). To resolve fine-grained local relationships, t-Distributed Stochastic Neighbor Embedding (t-SNE) was applied to a representative subsample. The visualizations effectively unfold the high-dimensional manifold, revealing that the dataset is composed of distinct topological structures rather than a uniform distribution. Consistent with the global analysis, the GMM projection (Figure 17) exhibits a strong topological congruence with the K-Means baseline (Figure 15). While the specific cluster indices (colors) differ due to initialization, both algorithms respect the same geometric boundaries described below (referencing K-Means IDs):

- **The Curvilinear Manifold (Cluster 1 and 4):** The most prominent feature is a large, curvilinear trajectory that dominates the projection. The main body of this structure is assigned to **Cluster 1**, which begins as a highly dense block, indicating a repetitive, low-variance attack pattern, before expanding into an arch-like formation. Crucially, the distal extremity (or "tail") of this arch is assigned to **Cluster 4**. The visual continuity between the dense Cluster 1 and the tapering Cluster 4 reinforces the hypothesis that Cluster 4 acts as a transitional or "staging" phase that naturally evolves into the specific campaign activity of Cluster 1.

Fig. 16. PCA projection of GMM clusters ($k = 5$).Fig. 17. t-SNE visualization of GMM clusters ($k = 5$).

- **The Isolated High-Density Island (Cluster 0):** In the bottom-left region, **Cluster 0** forms a strictly isolated, globular island. The extreme geometric compactness of this group suggests that the sessions within it are highly homogeneous, likely representing a specific, automated botnet script that executes with minimal variation.
- **Dispersed Regions (Cluster 3 and 2):** In contrast to the dense islands, the upper region of the manifold is occupied by **Cluster 3**, which exhibits a sparse, scattered distribution. This sparsity indicates high entropy, suggesting that Cluster 3 contains diverse or randomized command sequences. Finally, **Cluster 2** appears as a minor, distinct grouping in the center-right, spatially separated from the main manifold, confirming it as a unique behavioral outlier.

The fact that two mathematically distinct algorithms, one minimizing Euclidean variance (K-Means) and one maximizing Gaussian likelihood (GMM), produce such visually congruent manifolds confirms that these five clusters represent robust, real-world behavioral patterns intrinsic to the dataset.

4.4 Semantic Characterization of Clusters

To interpret the clustering results, we analyzed the dominant command patterns within the K-Means partition (Figure 18). (Detailed GMM profiling yielded congruent patterns and is reported in Appendix D).

- **Cluster 0 (Automated Botnets):** This group exhibits a highly repetitive vocabulary centered on IoT-targeting keywords like `gopalon`, `e8telnet`, and `caonimade`. The frequent use of writable directories (e.g., `/var/tmp/`) and environment unsetting identifies these sessions as scripted, non-interactive botnet workflows executed at scale.
- **Cluster 2 (Variant Execution):** Sharing the `gopalon` lexicon of Cluster 0, this group displays systematic variations in file naming and payload placement paths. These subtle differences suggest the use of parallel execution templates or distinct malware variants operating within the same infection ecosystem.
- **Cluster 1 (Context-Aware Reconnaissance):** Unlike the blind automation above, Cluster 1 prioritizes system fingerprinting. The frequent invocation of `/bin/busybox`, `cat /proc/cpuinfo`, and vendor-specific credentials (`huawei@123`) indicates that these attacks dynamically adjust to the host's architecture and identity.

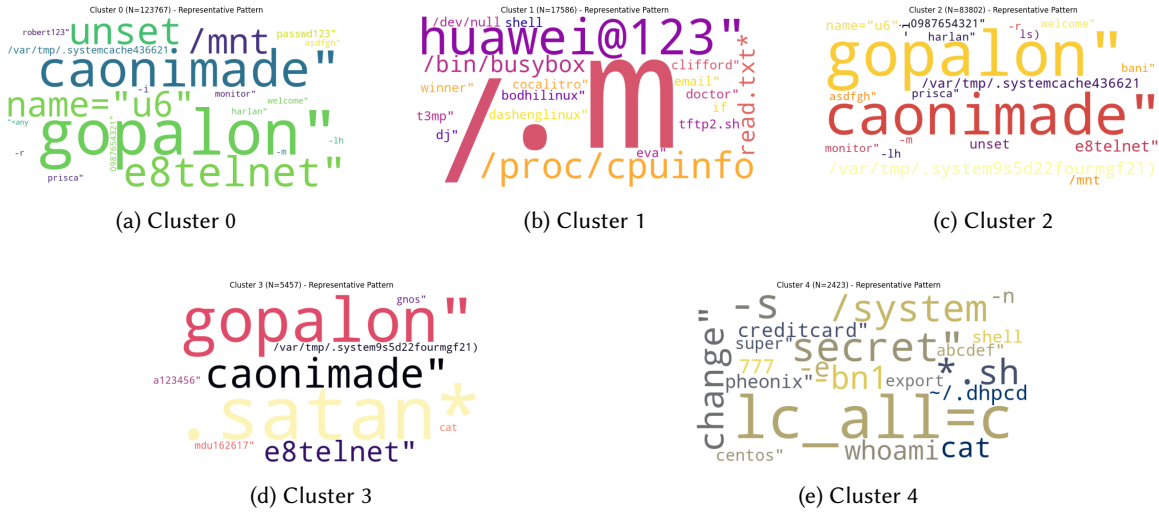


Fig. 18. Top frequent terms and commands for each K-Means cluster.

- **Cluster 3 (Stealth and Obfuscation):** Defined by technical sophistication, this cluster is dominated by Base64-encoded strings and markers like .satan. The commands focus on decoding and unpacking externally retrieved content, prioritizing stealth and evasion over active enumeration.
- **Cluster 4 (The "Staging" Phase):** Lacking unique signature keywords, this cluster aggregates generic administrative utilities (whoami, lc_all=c, chmod). As discussed in subsection 4.3, this heterogeneous mix functionally represents a "Staging" role, where standard commands prepare the environment for subsequent payloads.

4.5 External Evaluation: Homogeneity and Intent Division

To rigorously evaluate the quality of the discovered clusters, the partitions obtained from both **K-Means** and **GMM** were compared against the MITRE ATT&CK Ground Truth labels. Two complementary metrics were employed: **Purity**, measuring the extent to which a cluster contains sessions from a single tactical category, and the **Adjusted Rand Index (ARI)**, which quantifies the overall agreement between the clustering and the external labels while correcting for chance.

4.5.1 Quantitative Comparison. Table 8 presents the comparative results. The performance of both algorithms is virtually indistinguishable, reinforcing the validity of the discovered structure.

Table 8. Comparison of External Evaluation Metrics (K-Means vs. GMM)

Algorithm	Purity (0-1)	ARI (-1 to 1)
K-Means	0.9898	0.2213
GMM	0.9898	0.2210

The analysis reveals a noteworthy pattern consistent across both models:

- **High Purity** (≈ 0.99): This indicates that almost every cluster is internally homogeneous. The algorithms successfully isolated distinct behaviors without mixing conflicting MITRE tactics.
- **Low ARI** (≈ 0.22): The relatively low ARI indicates that the unsupervised clustering is much more granular than the Ground Truth. Instead of mapping 1-to-1 (e.g., 1 Cluster = 1 Tactic), the algorithms distinguish specific subgroups within the same Ground Truth category.

4.5.2 Intent Distribution. Figure 19 and Figure 20 illustrate the distribution of MITRE labels across the clusters for both K-Means and GMM. The visual alignment between the two methods is striking.

The granular nature of the unsupervised learning is evident in two key patterns:

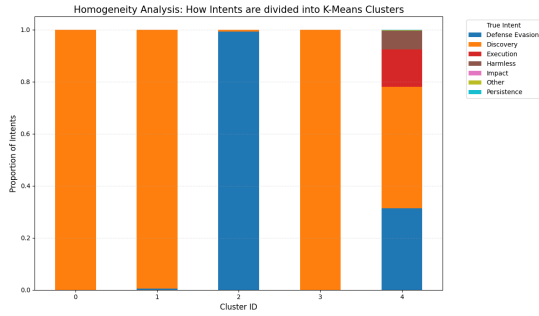


Fig. 19. Distribution of MITRE ATT&CK Ground Truth labels across K-Means clusters.

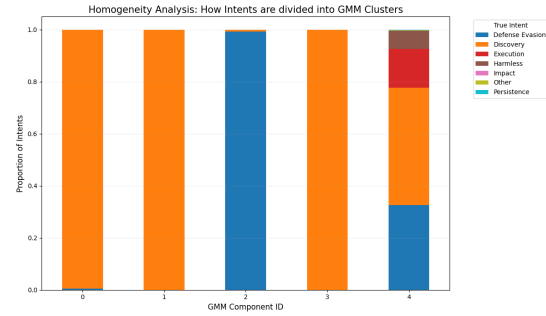


Fig. 20. Distribution of MITRE ATT&CK Ground Truth labels across K-Means clusters.

- **Fragmentation of "Discovery"**: The vast majority of sessions are labeled as Discovery in the Ground Truth. However, the algorithms do not group them into a single monolith. Instead, "Discovery" sessions are split across four distinct clusters (Clusters 0, 1, 3, and 4) in both K-Means and GMM. This confirms that the broad "Discovery" label actually encompasses multiple distinct attack campaigns (e.g., automated Mirai scanning vs. manual reconnaissance) that the models successfully disentangled.
- **The "Staging" Boundary (Cluster 4)**: Cluster 4 is the only heterogeneous group. In K-Means, the dominance of Discovery is only 46.65%, and in GMM it is 45.08%. The remaining proportion is a mix of other intents. This low dominance compared to the other clusters (which are >99% pure) confirms Cluster 4's role as a generic "Staging" or transitionary phase, containing commands that are common across multiple tactical goals.
- **Isolation of Defense Evasion**: Notably, Cluster 2 is the only group dominated by Defense Evasion (>99% in both models). This suggests that evasion techniques use a highly specific vocabulary that is mathematically distinct from the reconnaissance activities found in the other clusters.

These metrics demonstrate that the unsupervised clustering preserves homogeneity within groups while providing a finer resolution than the broad MITRE ATT&CK labels.

4.6 Fine-Grained Attack Categorization

To uncover specific attack campaigns and toolsets, this section integrates quantitative N-gram sequence mining and Co-occurrence Graphs with a qualitative manual inspection of raw logs. This multi-layered approach isolates distinct operational behaviors, revealing sophisticated patterns that extend beyond broad tactical classifications. This analysis focuses on the K-Means partition, as it serves as the baseline model for our study, though the behavioral insights derived here are equally applicable to the congruent GMM components.

Cluster 4 acts not as a heterogeneous error, but as a cohesive representation of a full Kill Chain attack cycle. The Cooccurrence Graph Figure 21 visualizes a robust semantic link between file transfer utilities and execution binaries, confirming this cluster represents the "Staging Phase" of an infection.

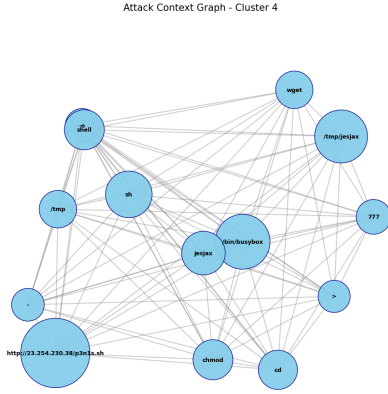


Fig. 21. Co-occurrence network of Cluster 4.

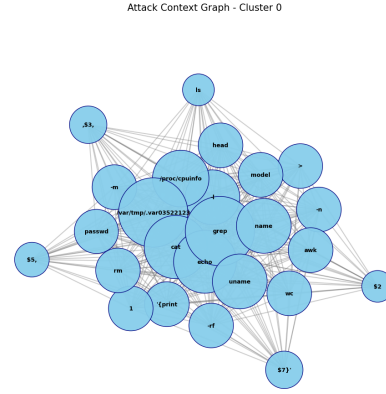


Fig. 22. Co-occurrence network of Cluster 0.

This is supported by N-gram analysis, which highlights a repetitive sequence of events: navigating to writable directories (`cd /tmp`), modifying permissions (`chmod 777`), and clearing environment variables (`lc_all=c`). Crucially, the manual inspection of raw logs revealed what statistical models missed: the extensive use of command chaining (e.g., `cd /tmp && wget ... && chmod +x`). Attackers use these single-line commands to automate the download-and-execute process, utilizing tools like `wget` to fetch payloads from remote servers.

The analysis of Cluster 0 provides the most significant insight. Although statistically dominated by *Discovery* labels, the fine-grained analysis exposes a sophisticated Resource Hijacking campaign.

The Co-occurrence Graph (Figure 22) links reconnaissance commands directly to system modification tools. While the most frequent N-grams appear harmless, focusing on hardware enumeration like `cat /proc/cpuinfo` or `grep`, the manual review discloses the true malicious logic.

These hardware scans are essentially "resource validation" steps immediately followed by a Competitive Account Takeover. The raw logs show syntax such as `echo -e '...' | passwd`, demonstrating that the malware validates the device's CPU power and then attempts to change the root password. This behavior is indicative of botnet turf wars: the attacker secures exclusive access to the device by locking out the legitimate owner and preventing rival botnets from exploiting the same host.

5 Language Models Exploration (BERT)

5.1 Introduction and Methodology

In this section, we move beyond traditional statistical embeddings to explore Large Language Models (LLMs), specifically applying BERT to the task of SSH command classification.

BERT represents a significant advancement in NLP due to its bidirectional nature. Unlike previous directional models, BERT utilizes the Transformer encoder architecture to learn the context of a word based on both its previous and next surroundings simultaneously. This capability is learned during pre-training via two unsupervised tasks: Masked Language Modeling (MLM) and Next Sentence Prediction (NSP).

Table 9. Overall Model Performance. Evaluation metrics on the test set showing near-perfect classification results.

Precision	Recall	F1 score	ROC AUC
0.9926	0.9915	0.9904	0.9959

5.2 Implementation Strategy: Transfer Learning

Since SSH commands exhibit complex syntactical structures similar to natural language, we hypothesize that BERT can capture contextual nuances that simpler models (like TF-IDF) might miss.

To implement this, we utilized Transfer Learning. We adopted the pre-trained bert-base-uncased model to serve as a high-level feature extractor. By freezing the internal weights of the BERT backbone, we retained its pre-learned linguistic knowledge and trained only a custom linear classification head. This allowed us to effectively map the rich, contextual embeddings generated by BERT to our specific multi-label cybersecurity intents.

5.3 Data Encoding and Loss Configuration

A critical challenge in this dataset is the **Multi-Label** nature of the target variable, where a single SSH session often corresponds to multiple simultaneous attack stages. To address this, we employed a `MultiLabelBinarizer` to map the intents into a dense Binary Indicator Matrix. This transformation allows the model to treat the prediction of each class as an independent binary classification problem, rather than a mutually exclusive choice.

To implement this, we modified the pre-trained BERT architecture by replacing the original classification layer with a custom **Linear Layer containing 7 neurons**. Critically, instead of the standard `CrossEntropyLoss` (which forces a single class selection via Softmax), we utilized **`BCEWithLogitsLoss`**. This function combines a **Sigmoid** activation layer and the **Binary Cross Entropy loss** into a single class, ensuring that the probability of each intent is calculated independently of the others.

The model was fine-tuned using the AdamW optimizer with a linear decay scheduler to preserve the pre-trained linguistic features. The complete training configuration, including hyperparameters, batch sizes, and learning rates, is detailed in Appendix E.

5.4 Model Performance Analysis

As shown in Table 9 the fine-tuned BERT model demonstrated exceptional performance on the test set, achieving a Global Weighted F1-Score of 0.9904 and an ROC AUC of 0.9959. These metrics indicate that the model has successfully learned to distinguish between complex command structures with near-perfect accuracy for the majority of attack intents.

5.4.1 Learning Dynamics and Convergence. The training process exhibited a healthy convergence profile. As observed in Figure 23, both training and validation loss decreased rapidly during the first few epochs.

- **Rapid Adaptation:** The most significant learning occurred within the first **5 epochs** (Validation loss dropping from 0.0595 to 0.0314).
- **Stability:** From Epoch 6 to 15, the improvement was marginal but steady (final Validation loss: 0.0226).
- **No Overfitting:** Crucially, the validation loss remained consistently lower than or equal to the training loss throughout the process. This suggests that the regularization techniques (Dropout, Weight Decay) were effective and that 15 epochs did not lead to memorization of the training data. The observation that significant learning concluded by **epoch 4** aligns with findings in the original BERT literature for datasets of this size.

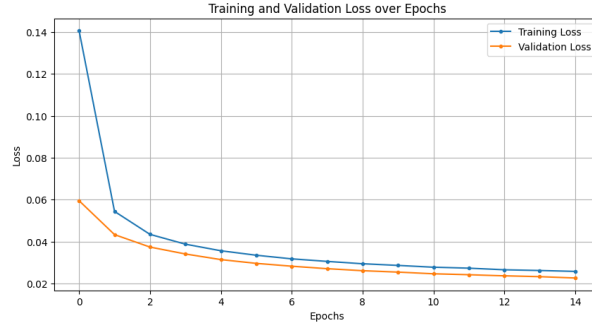


Fig. 23. Training and Validation Loss. Convergence stabilizes at Epoch 5 with no signs of overfitting, marking the optimal trade-off between performance and efficiency.

5.4.2 Class-Specific Performance and Robustness. While the global metrics are high, a granular analysis (detailed in subsection E.2) reveals distinct behaviors across intents. For dominant attack types (Discovery, Persistence, Defense Evasion), the model achieves near-perfect separation, with Precision and Recall exceeding 0.99. This confirms that BERT’s contextual embeddings successfully captured the syntax of these attacks. The ROC analysis further validates this robustness, yielding an AUC of 1.00 for all primary categories. The only notable limitation is observed in the Harmless class (Support: 458). As shown in the supplementary confusion matrices (Figure 31), the model exhibits a "paranoid" bias: it achieves high precision (0.97) but low recall (0.07), frequently misclassifying benign commands as malicious. In a cybersecurity context, this bias is operationally preferable, as false positive alerts are significantly less dangerous than missing a genuine intrusion.

6 Conclusion

While statistical models achieve near-perfect accuracy on this dataset, their reliance on static signatures makes them brittle against evasion. Our analysis reveals that simple metadata performs equivalently to TF-IDF, confirming the rigidity of current attack templates. In contrast, BERT offers superior robustness by capturing command context. Furthermore, unsupervised clustering successfully deconstructed generic tactics into specific botnet campaigns and a distinct "Staging" phase. Ultimately, integrating deep contextual embeddings with efficient statistical baselines is critical for resilient intrusion detection.

A Supplementary Data Distribution Analysis

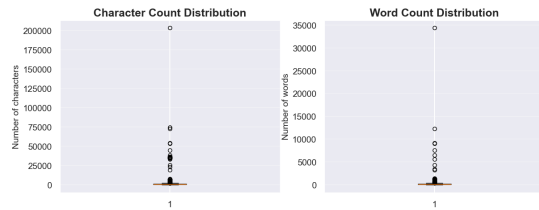


Fig. 24. Box Plots word and character count distribution.

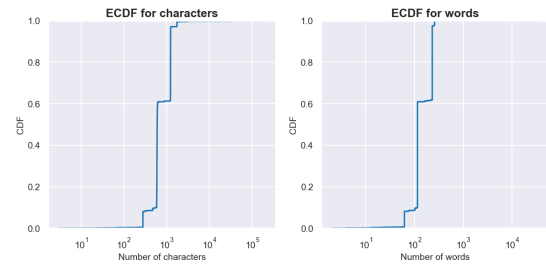


Fig. 25. Empirical Cumulative Distribution Function for characters and words per session.

The extreme variance within the dataset is formally confirmed through box plot analysis (Figure 24), which provides a clearer perspective on the system's "edge cases". While the previous histograms highlight the frequency of short sessions, the Character and Word count distribution in the box plots isolate the sheer distance of outliers from the median.

Furthermore, the Empirical Cumulative Distribution Function (ECDF) (Figure 25) provides a precise quantification of session saturation. Unlike a standard frequency count, the ECDF reveals the specific thresholds where the vast majority of activity is contained.

B Base64 String decoding

To illustrate the obfuscation techniques discussed in subsection 2.2, we analyze Session ID 758, where the original commands were hidden using Base64 encoding:

```
cd /var/tmp ;
echo "IyEvYm1uL2Jhc2gKY2QgL3RtcApybSAtcmYgLLgxNS11bm14Cm1rZGlyIC5YMTUtdW5peApjZCAu
WDE1LXVuaXgKcGtpbGwgLTkgY3JvbiA+IC5vdXQKd2dldCAtcSBodHRwOi8vNTQuMzcuNzAuMjQ5L2RvdG
EyLnRhci5neiB8fCBjdXJsIC1PIC1mIGh0dHA6Ly81NC4zNy43MC4yNDkvZG90YTIudGFyLmd6CnNsZWVw
IDdzICYmIHRhciB4ZiBkb3RhMi50YXIuZ3oKI3JtIC1yZiBkb3RhMi50YXIuZ3oKY2QgLnJzeW5jCmNobW
9kIDc3NyAqCmNkIC90bXAvLlgxNS11bm14Ly5yc3luYy9hICYmIC4vY3JvbiB8fCAuL2FuYWYyY24KZXhp
dCAw" | base64 --decode | bash ;
```

The last code segment is designed to execute the payload contained within the Base64 string. The command utilizes the `base64 --decode` flag to revert the obfuscated text to its original form, followed by a pipe `| bash` to immediately execute the resulting script. Upon manual decoding, the following Bash commands are revealed:

```
#!/bin/bash
cd /tmp
rm -rf .X15-unix
mkdir .X15-unix
cd .X15-unix
pkill -9 cron > .out
wget -q http://54.37.70.249/dota2.tar.gz || curl -O -f http://54.37.70.249/dota2.tar.gz
sleep 7s && tar xf dota2.tar.gz
#rm -rf dota2.tar.gz
cd .rsync
chmod 777 *
cd /tmp/.X15-unix/.rsync/a && ./cron || ./anacron
exit 0
```

Consequently, a function to decode all the Base64 encoded parts of the full session was created (`decode_base64_payloads`). The function utilizes a regular expression to scan the dataset for strings that follow the standard Base64 character set (A-Z, a-z, 0-9, +, /) and padding (=). To avoid false positives, a pragmatic constraint is applied that ignores any potential match shorter than 20 characters. Upon identifying a candidate string, the string attempts a strict Base64 decoding. If the resulting byte can be successfully converted into a UTF-8 string, the string proceeds to evaluate its readability. To distinguish between plain-text commands and raw binary data, the function calculates the ratio of non-printable characters. If the decoded content is more than 20% non-printable, it is flagged as binary and left in its original state to preserve data integrity.

C Supplementary Supervised Learning Analysis

C.1 Multilabel Misclassification Analysis

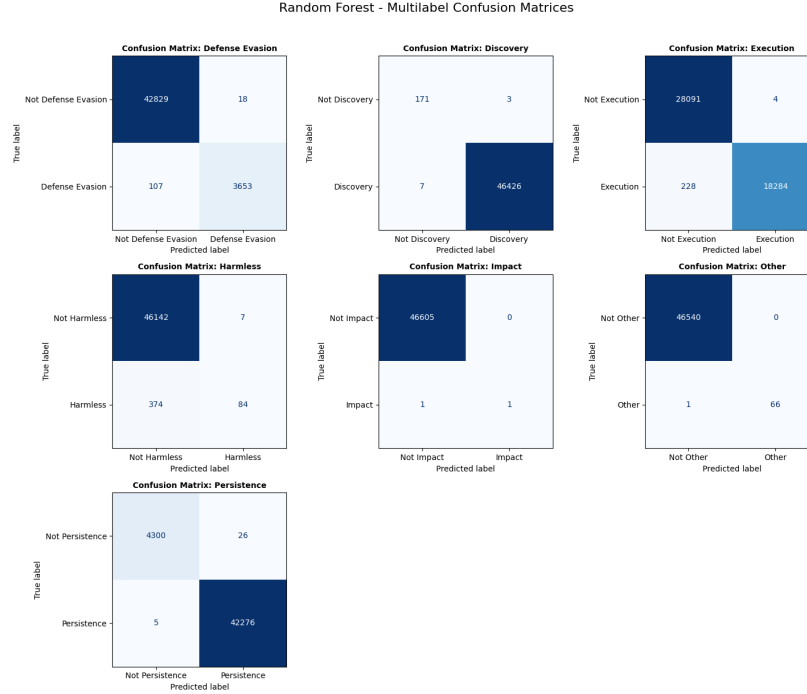


Fig. 26. Random Forest confusion matrices for each label

To provide a granular view of classification errors, specifically regarding the minority classes, we generated multilabel confusion matrices for both classifiers. Figure 26 (Random Forest) and Figure 27 (Logistic Regression) illustrate the True Positive and False Negative rates for each individual intent. These visualizations confirm that while dominant classes like *Discovery* and *Persistence* achieve near-perfect separation, the *Harmless* class suffers from significant False Negative rates across both classifiers. This visual evidence supports the low recall scores reported in Table 1 and Table 2, highlighting the difficulty of distinguishing benign commands from malicious ones in an imbalanced dataset.

C.2 Hyperparameter Tuning Visualization

To assess the impact of hyperparameter optimization, we visualized the Weighted F1-Score across the entire grid search space. Figure 28 (Random Forest) and Figure 29 (Logistic Regression) display the performance stability across different configurations. The uniformity of the heatmaps confirms that the models are highly robust, as increasing complexity (e.g., deeper trees or higher regularization) yielded negligible performance gains over the baseline defaults.

D Semantic characterization of GMM clusters

To validate the stability of the behavioral patterns identified by K-Means, a parallel semantic profiling was conducted on the clusters generated by the Gaussian Mixture Model (GMM). While the broad tactical categories

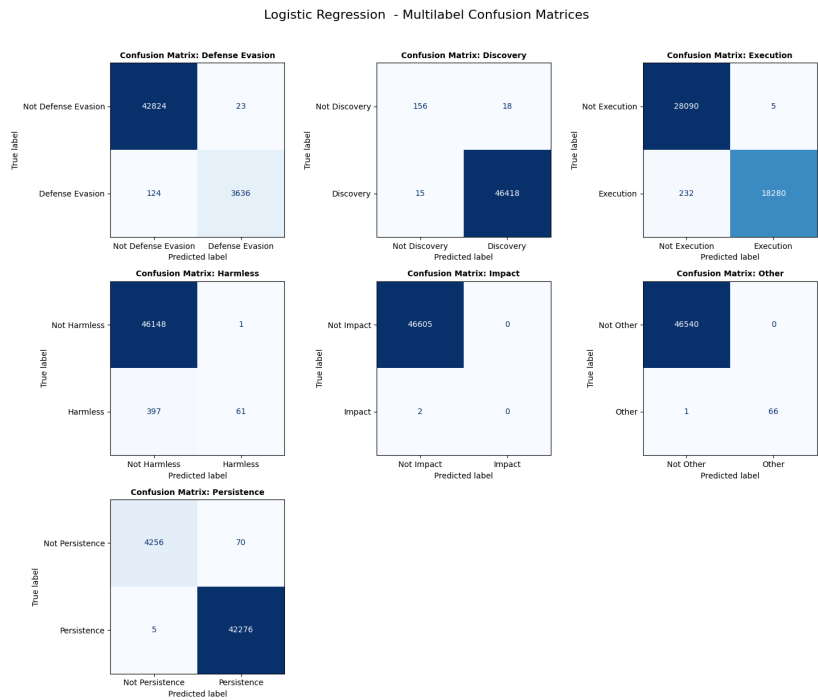


Fig. 27. Logistic Regression Multilabel confusion matrices

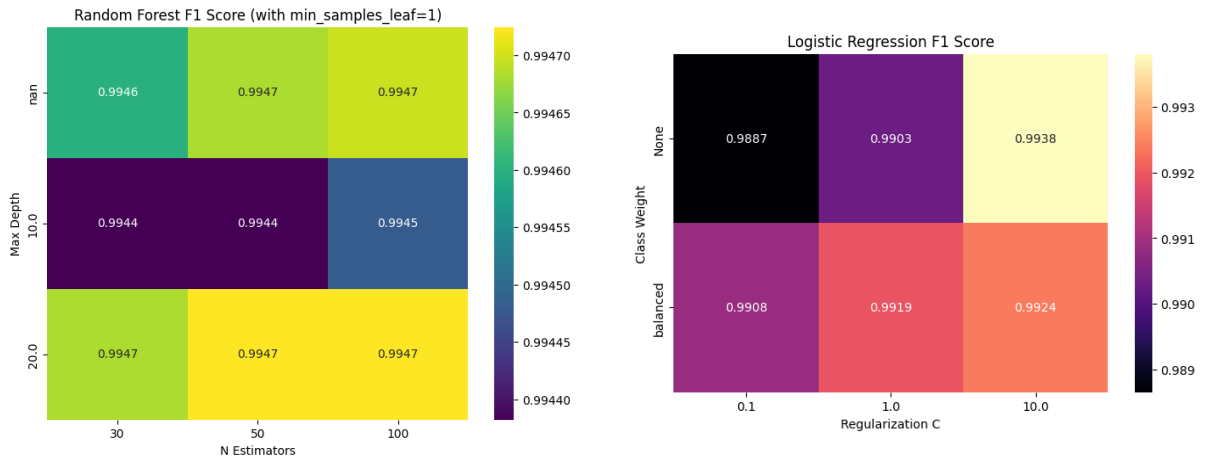


Fig. 28. Random Forest Grid Search F1 Score (Weighted) varying depth and estimators.

Fig. 29. Logistic Regression Grid Search F1 Score (Weighted) varying C and Class Weight.

align with the main analysis, the probabilistic nature of GMM highlighted specific campaign signatures and persistence mechanisms with greater granularity.

The GMM clusters exhibit distinct vocabularies that map to specific malware families and operational stages:

- **Cluster 0 (Legacy Botnets and Credentials):** Unlike the generic automation seen in K-Means, this cluster specifically isolates sessions containing keywords associated with known legacy botnets such as glacier and undernet. The presence of explicit credential strings (passw0rd1, asd123) and specific target identifiers (motorola, bodhilinux) suggests a campaign focused on credential stuffing against specific IoT architectures.
- **Cluster 1 (Network Reconnaissance - "Caleb/Dota"):** This group is defined by the recurring presence of the caleb and dota keywords, likely referring to specific malware payloads or actor handles. The co-occurrence of network flags such as -plnt (used with netstat) and retrieval commands suggests this cluster captures the network reconnaissance and payload staging phase of this specific campaign.
- **Cluster 2 (Persistence and System Locking):** This cluster represents the most technically distinct group, dominated by persistence and evasion mechanisms. Key terms include cron (job scheduling), chattr (file attribute locking to prevent removal), and references to hidden directory structures (.x17-unix, .x15cache). The inclusion of raspberry and whoami confirms a targeted effort to secure long-term access on embedded Linux devices.
- **Cluster 3 (Filesystem Staging - "Dota" Variant):** Sharing the dota signature with Cluster 1, this group focuses heavily on filesystem manipulation rather than network activity. The vocabulary is dominated by hidden directory paths (.x13-unix, .rsync) and payload extraction artifacts (.tar.gz), indicating the installation phase of the "Dota" malware variant.
- **Cluster 4 (Execution Scheduling):** This cluster aggregates generic execution and scheduling commands. Terms such as scheduler, .t, and client, alongside specific campaign markers like hooters and diamond, point to the final execution phase where the malware establishes its client connection to the C2 server.

Figure 30 presents the word clouds for the GMM clusters, visually confirming the segregation of specific botnet campaigns (Glacier, Dota) and functional tactics (*Persistence, Reconnaissance*).

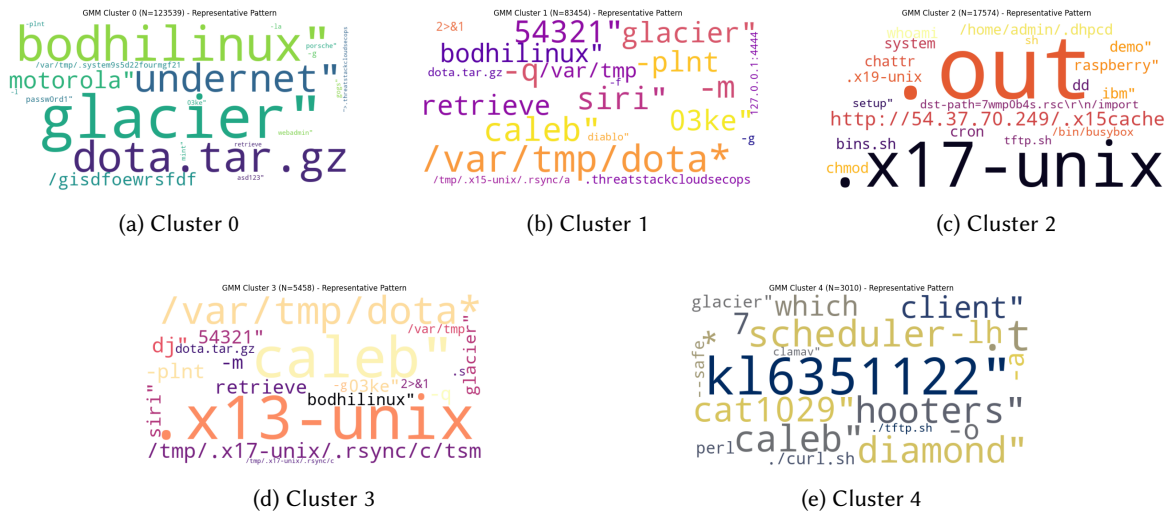


Fig. 30. Top frequent terms and commands for each GMM cluster.

E Supplementary BERT Analysis

E.1 BERT Training Configuration

The BERT model was fine-tuned using the following specific configuration, selected to balance convergence speed with memory stability on the GPU:

- **Optimizer:** AdamW (Adam with Weight Decay) was selected to decouple weight decay from the gradient update.
- **Learning Rate:** Initial rate of 4×10^{-5} with a Linear Decay scheduler reducing to 0 over the course of training.
- **Batch Size:** 16 (aligned with original BERT recommendations).
- **Epochs:** 15. Although the original BERT literature recommends 2–4 epochs for fine-tuning, we extended this to 15 to ensure full convergence. Our validation monitoring confirmed the standard recommendation, as the most significant learning occurred within the first 5 epochs without signs of overfitting (Figure 23).
- **Regularization:** Standard BERT dropout and weight decay were applied to prevent memorization of the training data.

E.2 Detailed Performance Visualization

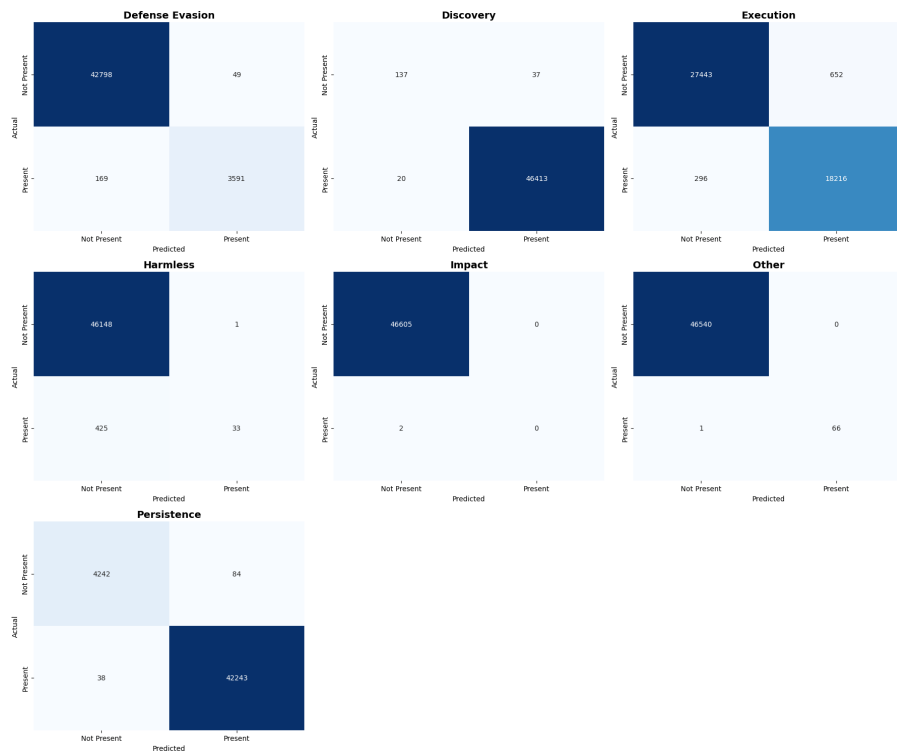


Fig. 31. Per-Class Confusion Matrices. Visualization of True Positives and False Negatives, demonstrating high precision for all classes except the under-represented *Harmless* category.

To granularly analyze the model's behavior, we generated class-specific performance metrics.

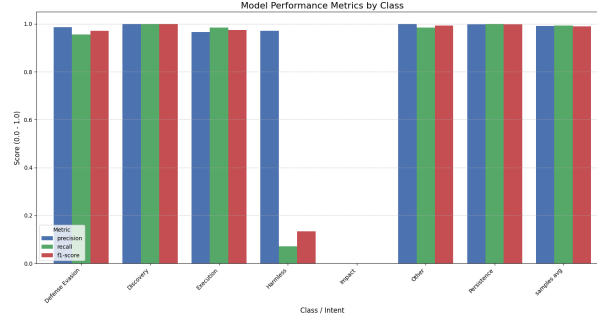


Fig. 32. Precision, Recall, and F1-Score by Intent. Visualizing the high effectiveness of the model on the primary attack classes compared to the minority *Harmless* and *Impact* classes.

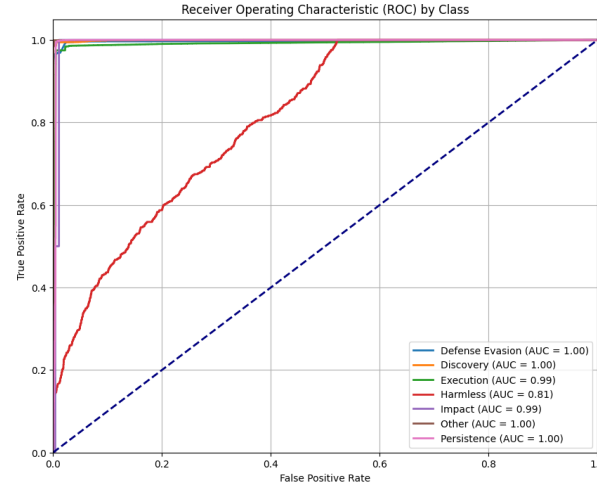


Fig. 33. Per-Class ROC Analysis. Almost all classes show ideal separation (AUC 1.0), with only the minority 'Harmless' class dropping to 0.81.

- **Confusion Matrices (Figure 31):** Visualize the True Positives and False Negatives, demonstrating near-perfect precision for dominant classes while highlighting the recall challenges in the minority Harmless category.
- **Classification Report (Figure 32):** Visualizes the Precision, Recall, and F1-Score breakdown by intent.
- **ROC Curves (Figure 33):** Validates the model's robustness, with AUC values reaching 1.00 for all major attack vectors, dropping only to 0.81 for the Harmless class.